

EXPERIMENT NO. 2

Aim

To convert the provided Indian E-Commerce platform Entity-Relationship (ER) diagram into a normalized relational schema, write complete MySQL CREATE TABLE DDL statements with primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, ON DELETE CASCADE, and ON DELETE SET NULL constraints, insert structured sample records representing production scenarios, and demonstrate referential integrity violations, constraint exceptions, and cascade/set-null actions.

Theory

Relational Schema

A relational schema is the logical, table-based representation of a database, obtained by systematically converting an ER diagram into a set of relations (tables), each with defined columns, data types, domain rules, and constraints. This conversion process is called ER-to-Relational Mapping.

The ER model captures business rules conceptually (entities, relationships, cardinalities, constraints), but a relational database management system (RDBMS) can only store and query data structured in rectangular tables composed of tuples and attributes. This mapping process translates that conceptual design into an implemented relational schema while preserving all the original semantics: keys stay unique, weak entities stay dependent, multi-valued attributes are isolated, and relationship constraints are enforced automatically by the database engine through declarative constraints defined in SQL statements.

Rules for Mapping ER Diagram to Relational Schema

- 1. Strong Entity:** Becomes a distinct table; its key attribute becomes the primary key of the table (e.g., in our ER diagram: Customer, Seller, Category, Product, Order, Payment, Delivery).
- 2. Weak Entity:** Becomes a table whose primary key is a composite key formed by the owner entity's primary key together with the weak entity's own partial discriminator key. A foreign key referencing the owner is defined, typically paired with ON DELETE CASCADE, since the weak entity cannot exist without its identifying owner (e.g., Address depends on Customer; OrderItem depends on Order).
- 3. 1:1 Relationship:** The foreign key can be placed on either participating entity's table (preferably on the table with total participation) and marked UNIQUE, so each row on that side maps to at most one row on the opposite side (e.g., Order – Payment, Order – Delivery).
- 4. 1:N Relationship:** The foreign key is placed in the table corresponding to the entity on the “many” side, referencing the primary key of the entity on the “one” side (e.g., Customer places Order; Seller lists Product; Category classifies Product).
- 5. M:N Relationship:** A new junction/bridge table (associative relation) is created containing foreign keys referencing both participating entities. The primary key of the bridge table is composite, containing both

foreign keys along with any descriptive attributes belonging to the relationship (e.g., OrderItem resolves the many-to-many relationship between Order and Product).

6. Multi-Valued Attribute: Becomes a separate child table containing a foreign key referencing the owning entity and a column to hold the attribute value. A compound primary key consisting of the owning key and the attribute value ensures 1NF compliance, since a single relational column cannot hold multiple values or repeating groups (e.g., Customer phone_numbers, Seller contact_numbers, Product images, Delivery status_history).

7. Composite Attribute: Its sub-parts become individual columns in the parent table (flattening); the composite concept itself is not stored as an independent type (e.g., name → first_name, last_name; dimensions → length, breadth, height).

8. Specialization / is_a Hierarchy: Each subclass becomes its own table sharing the primary key of the superclass table, linked via a foreign key with ON DELETE CASCADE. A type discriminator column may be preserved in the superclass table to indicate which subclass a given instance belongs to (e.g., Product → Electronics, Clothing, Grocery).

9. Recursive / Self-Referencing Relationship: The foreign key resides in the same table and references the table's own primary key (e.g., Category's parent_category_id references Category(category_id)).

Integrity Constraints Used in the Schema

- **Primary Key Constraints:** Uniquely identify each row in a relation; enforce entity integrity by rejecting NULL values and duplicate entries.
- **Foreign Key (Referential Integrity) Constraints:** Ensure that a value appearing in one table matches an existing primary key value in another table, preventing dangling references and orphaned child records.
- **NOT NULL Constraints:** Force a column to always have an explicit scalar value, reflecting total participation or non-optional business attributes.
- **UNIQUE Constraints:** Ensure no duplicate values exist across a column or column group (e.g., email, gstin, tracking tokens), and enforce 1:1 cardinality when defined on a foreign key.
- **CHECK Constraints:** Enforce domain-specific validation rules (e.g., valid 15-digit Indian GSTIN formats, 10-digit mobile numbers, non-negative prices and quantities).

Referential Actions (ON DELETE / ON UPDATE):

- **CASCADE:** Automatically deletes or updates dependent child rows when the parent row is deleted or updated. Used for weak entities, multi-valued tables, and specialization subtypes that have no independent existence.
 - **SET NULL:** Sets the referencing foreign key to NULL when the referenced parent row is deleted. Used when the relationship is optional, allowing the dependent row to survive without the parent (e.g., a product surviving if its parent category is retired).
 - **RESTRICT:** Rejects deletion of the parent row as long as referencing records exist.
-

Step-by-Step Relational Schema Derivation

CONCEPTUAL ER SCHEME	
Customer	(cust_id, name(first, last), email, {phone_numbers})
Address	[[cust_id, address_type, street, city]]
Seller	(seller_id, gstin, {contact_numbers})
Category	(cat_id, cat_name, parent_cat_id)
Product	(prod_id, name, dimensions(len, brd, hgt), price, {images})
	--> Electronics [[prod_id, warranty_months, power_watts]]
	--> Clothing [[prod_id, fabric, size, gender]]
	+--> Grocery [[prod_id, expiry_date, is_veg]]
Order	(order_id, cust_id, order_date, status, [order_total])
OrderItem	[[order_id, prod_id, quantity, unit_price, [line_total]]]
Payment	(payment_id, order_id, payment_mode, status, amount)
Delivery	(delivery_id, order_id, courier, tracking_num, {status_history})

Based on the mapping rules, the 11 entities, multi-valued attributes, and relationships present in the diagram map to 16 normalized relational tables:

1. customers (customer_id, first_name, last_name, email, created_at)
 2. customer_phone_numbers (customer_id, phone_number)
 3. addresses (address_id, customer_id, address_type, street, city)
 4. sellers (seller_id, legal_name, gstin, state)
 5. seller_contact_numbers (seller_id, contact_number)
 6. categories (category_id, category_name, parent_category_id)
 7. products (product_id, category_id, seller_id, product_name, length, breadth, height, base_price)
 8. product_images (image_id, product_id, image_url)
 9. electronics_products (product_id, warranty_period_months, power_consumption_watts)
 10. clothing_products (product_id, fabric_type, size_code, gender_fit)
 11. grocery_products (product_id, expiry_date, nutritional_info, is_veg)
 12. orders (order_id, customer_id, order_date, status)
 13. order_items (order_item_id, order_id, product_id, quantity, unit_price, line_total)
 14. payments (payment_id, order_id, payment_mode, transaction_status, amount)
 15. deliveries (delivery_id, order_id, courier_name, tracking_number)
 16. delivery_status_history (history_id, delivery_id, status, status_timestamp, location)
-

Source Code: Complete MySQL DDL Implementation

```
-- =====
-- EXPERIMENT 02: INDIAN E-COMMERCE PLATFORM RELATIONAL SCHEMA
-- Database Engine: MySQL 8.0+
-- =====

DROP DATABASE IF EXISTS indian_ecommerce_db;
CREATE DATABASE indian_ecommerce_db;
USE indian_ecommerce_db;

-- -----
-- 1. CUSTOMER TABLE (Strong Entity with Flattened Name Attribute)
-- -----
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- -----
-- 2. CUSTOMER PHONE NUMBERS TABLE (Multi-Valued Attribute of Customer)
-- -----
CREATE TABLE customer_phone_numbers (
    customer_id INT NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    PRIMARY KEY (customer_id, phone_number),
    CONSTRAINT chk_cust_phone_format CHECK (phone_number REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_cust_phone_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;

-- -----
-- 3. ADDRESS TABLE (Weak Entity Dependent on Customer)
-- -----
CREATE TABLE addresses (
    address_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    address_type VARCHAR(20) NOT NULL, -- 'Home', 'Office', 'Billing'
    street VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    CONSTRAINT fk_address_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT uq_customer_address UNIQUE (customer_id, address_type, street)
) ENGINE=InnoDB;

-- -----
-- 4. SELLER TABLE (Strong Entity with GSTIN Identifier)
-- -----
CREATE TABLE sellers (
    seller_id INT PRIMARY KEY AUTO_INCREMENT,
    legal_name VARCHAR(150) NOT NULL,
    gstin CHAR(15) NOT NULL UNIQUE,
    state VARCHAR(50) NOT NULL,
    CONSTRAINT chk_gstin_format CHECK (gstin REGEXP '^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$')
) ENGINE=InnoDB;

-- -----
-- 5. SELLER CONTACT NUMBERS TABLE (Multi-Valued Attribute of Seller)
-- -----
CREATE TABLE seller_contact_numbers (
```

```

        seller_id INT NOT NULL,
        contact_number VARCHAR(15) NOT NULL,
        PRIMARY KEY (seller_id, contact_number),
        CONSTRAINT fk_seller_contact_seller
            FOREIGN KEY (seller_id)
            REFERENCES sellers(seller_id)
            ON DELETE CASCADE
            ON UPDATE CASCADE
    ) ENGINE=InnoDB;

```

```

-----
-- 6. CATEGORY TABLE (Self-Referencing Unary Hierarchy)
-----

```

```

CREATE TABLE categories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    parent_category_id INT NULL,
    CONSTRAINT fk_cat_parent
        FOREIGN KEY (parent_category_id)
        REFERENCES categories(category_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
) ENGINE=InnoDB;

```

```

-----
-- 7. PRODUCT TABLE (Strong Entity with Flattened Dimensions)
-----

```

```

CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    category_id INT NULL,
    seller_id INT NOT NULL,
    product_name VARCHAR(200) NOT NULL,
    length DECIMAL(6, 2) NOT NULL, -- Flattened dimensions component
    breadth DECIMAL(6, 2) NOT NULL, -- Flattened dimensions component
    height DECIMAL(6, 2) NOT NULL, -- Flattened dimensions component
    base_price DECIMAL(10, 2) NOT NULL,
    CONSTRAINT chk_prod_price CHECK (base_price >= 0.00),
    CONSTRAINT fk_product_category
        FOREIGN KEY (category_id)
        REFERENCES categories(category_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE,
    CONSTRAINT fk_product_seller
        FOREIGN KEY (seller_id)
        REFERENCES sellers(seller_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
) ENGINE=InnoDB;

```

```

-----
-- 8. PRODUCT IMAGES TABLE (Multi-Valued Attribute of Product)
-----

```

```

CREATE TABLE product_images (
    image_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    image_url VARCHAR(255) NOT NULL,
    CONSTRAINT fk_prod_image_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;

```

```

-----
-- 9. SPECIALIZATION SUBCLASS: ELECTRONICS PRODUCTS (is_a Product)
-----

```

```

CREATE TABLE electronics_products (
    product_id INT PRIMARY KEY,
    warranty_period_months INT NOT NULL DEFAULT 12,
    power_consumption_watts DECIMAL(6, 2) NOT NULL,
    CONSTRAINT fk_elec_product

```

```

        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

-----
-- 10. SPECIALIZATION SUBCLASS: CLOTHING PRODUCTS (is_a Product)
-----
CREATE TABLE clothing_products (
    product_id INT PRIMARY KEY,
    fabric_type VARCHAR(50) NOT NULL,
    size_code VARCHAR(10) NOT NULL,
    gender_fit VARCHAR(20) NOT NULL,
    CONSTRAINT fk_cloth_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

-----
-- 11. SPECIALIZATION SUBCLASS: GROCERY PRODUCTS (is_a Product)
-----
CREATE TABLE grocery_products (
    product_id INT PRIMARY KEY,
    expiry_date DATE NOT NULL,
    nutritional_info TEXT NULL,
    is_veg BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_groc_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
    ) ENGINE=InnoDB;

-----
-- 12. ORDER TABLE (Strong Entity; Places Relationship with Customer)
-----
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) NOT NULL DEFAULT 'PLACED',
    CONSTRAINT fk_order_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT chk_order_status CHECK (status IN ('PLACED', 'CONFIRMED', 'SHIPPED', 'DELIVERED',
    'CANCELLED'))
    ) ENGINE=InnoDB;

-----
-- 13. ORDERITEM TABLE (Weak Entity / Junction Resolving Order-Product M:N)
-----
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    line_total DECIMAL(10, 2) GENERATED ALWAYS AS (quantity * unit_price) STORED,
    CONSTRAINT chk_oi_qty CHECK (quantity > 0),
    CONSTRAINT chk_oi_price CHECK (unit_price >= 0.00),
    CONSTRAINT fk_oi_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_oi_product

```

```

        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT uq_order_product UNIQUE (order_id, product_id)
) ENGINE=InnoDB;

-----
-- 14. PAYMENT TABLE (1:1 Relationship with Order)
-----
CREATE TABLE payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL UNIQUE, -- UNIQUE enforces strict 1:1 relationship
    payment_mode VARCHAR(30) NOT NULL,
    transaction_status VARCHAR(20) NOT NULL DEFAULT 'SUCCESS',
    amount DECIMAL(12, 2) NOT NULL,
    CONSTRAINT chk_pay_amount CHECK (amount >= 0.00),
    CONSTRAINT chk_pay_mode CHECK (payment_mode IN ('UPI', 'CREDIT_CARD', 'DEBIT_CARD',
'NET_BANKING', 'COD')),
    CONSTRAINT fk_payment_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;

-----
-- 15. DELIVERY TABLE (1:1 Relationship with Order)
-----
CREATE TABLE deliveries (
    delivery_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL UNIQUE, -- UNIQUE enforces strict 1:1 relationship
    courier_name VARCHAR(100) NOT NULL,
    tracking_number VARCHAR(100) NOT NULL UNIQUE,
    CONSTRAINT fk_delivery_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;

-----
-- 16. DELIVERY STATUS HISTORY (Multi-Valued Lifecycle Attribute of Delivery)
-----
CREATE TABLE delivery_status_history (
    history_id INT PRIMARY KEY AUTO_INCREMENT,
    delivery_id INT NOT NULL,
    status VARCHAR(50) NOT NULL,
    status_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    location VARCHAR(100) NOT NULL,
    CONSTRAINT fk_history_delivery
        FOREIGN KEY (delivery_id)
        REFERENCES deliveries(delivery_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;

```

Source Code: Population Scripts (DML Insertion)

```

-- =====
-- INSERTION OF VALID SAMPLE DATA
-- =====

-- 1. Insert Customers
INSERT INTO customers (customer_id, first_name, last_name, email) VALUES

```

```

(1, 'Aarav', 'Patel', 'aarav.patel@example.in'),
(2, 'Diya', 'Nair', 'diya.nair@example.in'),
(3, 'Vikram', 'Iyer', 'vikram.iyer@example.in');

-- 2. Insert Customer Phone Numbers (Multi-Valued)
INSERT INTO customer_phone_numbers (customer_id, phone_number) VALUES
(1, '9820112233'),
(1, '9820998877'),
(2, '9447112233'),
(3, '9845012345');

-- 3. Insert Customer Addresses (Weak Entity Instances)
INSERT INTO addresses (address_id, customer_id, address_type, street, city) VALUES
(1, 1, 'Home', 'Flat 402, Green Heights, Powai', 'Mumbai'),
(2, 1, 'Office', 'Level 4, Cyber City, Bandra', 'Mumbai'),
(3, 2, 'Home', 'Villa 12, Indiranagar', 'Bengaluru'),
(4, 3, 'Home', 'Plot 55, Anna Nagar', 'Chennai');

-- 4. Insert Sellers
INSERT INTO sellers (seller_id, legal_name, gstin, state) VALUES
(1, 'RetailNet India LLP', '27AABCR1234F1Z5', 'Maharashtra'),
(2, 'Surat Fabric House', '24AAECS9876M1Z2', 'Gujarat'),
(3, 'South Agro Organics', '33AACCA4433P1Z9', 'Tamil Nadu');

-- 5. Insert Seller Contacts (Multi-Valued)
INSERT INTO seller_contact_numbers (seller_id, contact_number) VALUES
(1, '022-28765432'),
(1, '022-28765433'),
(2, '0261-2345678'),
(3, '044-28114455');

-- 6. Insert Categories (Hierarchical with Recursive Self-Reference)
INSERT INTO categories (category_id, category_name, parent_category_id) VALUES
(1, 'Electronics', NULL),
(2, 'Clothing', NULL),
(3, 'Grocery', NULL),
(4, 'Audio Devices', 1),
(5, 'Ethnic Wear', 2);

-- 7. Insert Products (Composite Dimensions Flattened)
INSERT INTO products (product_id, category_id, seller_id, product_name, length, breadth, height,
base_price) VALUES
(101, 4, 1, 'Noise Cancelling Headphones', 18.00, 15.00, 7.50, 7999.00),
(102, 5, 2, 'Handspun Khadi Kurta', 75.00, 48.00, 2.00, 1499.00),
(103, 3, 3, 'Filter Coffee Decoction 500ml', 10.00, 8.00, 22.00, 350.00);

-- 8. Insert Product Images (Multi-Valued)
INSERT INTO product_images (image_id, product_id, image_url) VALUES
(1, 101, 'https://img.cdn.in/p101_angle1.jpg'),
(2, 101, 'https://img.cdn.in/p101_angle2.jpg'),
(3, 102, 'https://img.cdn.in/p102_front.jpg'),
(4, 103, 'https://img.cdn.in/p103_pack.jpg');

-- 9. Insert Specializations (is_a Product)
INSERT INTO electronics_products (product_id, warranty_period_months, power_consumption_watts)
VALUES
(101, 24, 2.50);

INSERT INTO clothing_products (product_id, fabric_type, size_code, gender_fit) VALUES
(102, 'Khadi Cotton', 'XL', 'Men');

INSERT INTO grocery_products (product_id, expiry_date, nutritional_info, is_veg) VALUES
(103, '2027-06-30', 'Zero added sugar, 100% Arabica', TRUE);

-- 10. Insert Orders
INSERT INTO orders (order_id, customer_id, order_date, status) VALUES
(1001, 1, '2026-09-01 10:30:00', 'DELIVERED'),
(1002, 2, '2026-09-02 14:15:00', 'SHIPPED'),
(1003, 3, '2026-09-03 18:00:00', 'PLACED');

```

```

-- 11. Insert OrderItems (Resolves M:N; Computes Stored line_total)
INSERT INTO order_items (order_item_id, order_id, product_id, quantity, unit_price) VALUES
(1, 1001, 101, 1, 7999.00),
(2, 1001, 103, 2, 350.00),
(3, 1002, 102, 3, 1499.00),
(4, 1003, 103, 4, 350.00);

-- 12. Insert Payments (1:1 with Orders)
INSERT INTO payments (payment_id, order_id, payment_mode, transaction_status, amount) VALUES
(501, 1001, 'UPI', 'SUCCESS', 8699.00),
(502, 1002, 'CREDIT_CARD', 'SUCCESS', 4497.00),
(503, 1003, 'COD', 'SUCCESS', 1400.00);

-- 13. Insert Deliveries (1:1 with Orders)
INSERT INTO deliveries (delivery_id, order_id, courier_name, tracking_number) VALUES
(901, 1001, 'BlueDart Express', 'BD-IN-9821334'),
(902, 1002, 'Delhivery Logistics', 'DEL-IN-4455112');

-- 14. Insert Delivery Status History (Multi-Valued Tracking Audit)
INSERT INTO delivery_status_history (history_id, delivery_id, status, status_timestamp, location)
VALUES
(1, 901, 'Order Packed', '2026-09-01 12:00:00', 'Warehouse Bhiwandi, MH'),
(2, 901, 'Out for Delivery', '2026-09-02 08:30:00', 'Hub Powai, Mumbai'),
(3, 901, 'Delivered', '2026-09-02 14:00:00', 'Customer Doorstep, Mumbai'),
(4, 902, 'Manifest Generated', '2026-09-02 16:00:00', 'Hub Surat, GJ'),
(5, 902, 'In Transit', '2026-09-03 06:00:00', 'Interstate Hub Pune, MH');

```

Analytical Table Verification and Display Scripts

```

-- DISPLAY CUSTOMER DIRECTORY WITH PHONE CONTACTS
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    c.email,
    GROUP_CONCAT(cp.phone_number SEPARATOR ', ') AS registered_phones
FROM customers c
LEFT JOIN customer_phone_numbers cp ON c.customer_id = cp.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email;

-- DISPLAY PRODUCT CATALOG WITH SPECIALIZATION ATTRIBUTES
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    s.legal_name AS seller_name,
    p.base_price,
    CONCAT(p.length, ' x ', p.breadth, ' x ', p.height, ' cm') AS dimensions_lxwxh,
    COALESCE(CONCAT('Warranty: ', ep.warranty_period_months, 'M, Power: ',
ep.power_consumption_watts, 'W'),
    CONCAT('Fabric: ', cp.fabric_type, ', Size: ', cp.size_code, ', Fit: ', cp.gender_fit),
    CONCAT('Expiry: ', gp.expiry_date, ', Veg: ', gp.is_veg)) AS specialized_properties
FROM products p
LEFT JOIN categories c ON p.category_id = c.category_id
LEFT JOIN sellers s ON p.seller_id = s.seller_id
LEFT JOIN electronics_products ep ON p.product_id = ep.product_id
LEFT JOIN clothing_products cp ON p.product_id = cp.product_id
LEFT JOIN grocery_products gp ON p.product_id = gp.product_id;

-- DISPLAY COMPLETE ORDER BREAKDOWN INCLUDING DERIVED TOTALS
SELECT
    o.order_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,

```

```

        oi.quantity,
        oi.unit_price,
        oi.line_total,
        pay.payment_mode,
        pay.amount AS total_order_amount,
        d.courier_name,
        d.tracking_number,
        o.status AS order_status
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN payments pay ON o.order_id = pay.order_id
LEFT JOIN deliveries d ON o.order_id = d.order_id;

```

Referential Integrity and Constraint Demonstrations

```

-- =====
-- DEMONSTRATING CONSTRAINT VIOLATIONS & INTEGRITY ENFORCEMENT
-- =====

-- -----
-- (A) INVALID FOREIGN KEY INSERTION
-- Inserting an order with a non-existent customer_id (e.g., 99)
-- -----
INSERT INTO orders (order_id, customer_id, status)
VALUES (2001, 99, 'PLACED');

-- -----
-- (B) UNIQUE CONSTRAINT VIOLATION
-- Inserting a customer with an email that already exists ('aarav.patel@example.in')
-- -----
INSERT INTO customers (first_name, last_name, email)
VALUES ('Aarav', 'Sharma', 'aarav.patel@example.in');

-- -----
-- (C) COMPOSITE PRIMARY KEY VIOLATION ON WEAK/JUNCTION RELATION
-- Inserting duplicate (order_id, product_id) into order_items
-- -----
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (1001, 101, 2, 7999.00);

-- -----
-- (D) STRICT 1:1 CARDINALITY VIOLATION
-- Inserting a second payment for an order that is already paid
-- -----
INSERT INTO payments (order_id, payment_mode, transaction_status, amount)
VALUES (1001, 'NET_BANKING', 'SUCCESS', 8699.00);

-- -----
-- (E) ON DELETE SET NULL DEMONSTRATION
-- Deleting parent category 1 ('Electronics'); the child category 'Audio Devices'
-- (via categories.parent_category_id) and any product directly assigned to
-- category 1 (via products.category_id) must survive with their foreign key
-- columns set to NULL, instead of blocking the delete or leaving dangling references.
-- -----
-- No product in the base dataset is directly assigned to category 1 (only to its
-- child, category 4), so insert one demonstration product to exercise fk_product_category:
INSERT INTO products (product_id, category_id, seller_id, product_name, length, breadth, height,
base_price)
VALUES (104, 1, 1, 'Demo Category-Linked Gadget', 10.00, 10.00, 5.00, 999.00);

-- Before delete: verify existing associations
SELECT product_id, product_name, category_id FROM products WHERE category_id IN (1, 4);

```

```
SELECT category_id, category_name, parent_category_id FROM categories WHERE parent_category_id = 1;

-- Perform deletion of parent Category 1
DELETE FROM categories WHERE category_id = 1;

-- After delete: check that both the child category and the linked product survived
-- with their foreign keys set to NULL
SELECT category_id, category_name, parent_category_id FROM categories WHERE category_id = 4;
SELECT product_id, product_name, category_id FROM products WHERE product_id = 104;

-----
-- (F) ON DELETE CASCADE DEMONSTRATION
-- Deleting customer 1 ('Aarav Patel'); associated rows in customer_phone_numbers
-- and addresses must be automatically purged by the database engine.
-----

-- Count dependent records before deletion
SELECT COUNT(*) AS pre_delete_phones FROM customer_phone_numbers WHERE customer_id = 1;
SELECT COUNT(*) AS pre_delete_addresses FROM addresses WHERE customer_id = 1;

-- Delete Customer 1 (Note: must not be blocked if orders are decoupled or cascaded)
-- For demonstration, create an isolated customer and cascade delete them:
INSERT INTO customers (customer_id, first_name, last_name, email)
VALUES (9, 'Test', 'User', 'temp.test@example.in');
INSERT INTO customer_phone_numbers (customer_id, phone_number) VALUES (9, '9999988888');
INSERT INTO addresses (customer_id, address_type, street, city) VALUES (9, 'Home', 'Temporary Lane',
'Pune');

-- Perform cascade delete
DELETE FROM customers WHERE customer_id = 9;

-- Verify cascade execution
SELECT * FROM customer_phone_numbers WHERE customer_id = 9;
SELECT * FROM addresses WHERE customer_id = 9;
```

Output

Output 1: Customer Directory with Normalized Multi-Valued Phones

customer_id	full_name	email	registered_phones
1	Aarav Patel	aarav.patel@example.in	9820112233, 9820998877
2	Diya Nair	diya.nair@example.in	9447112233
3	Vikram Iyer	vikram.iyer@example.in	9845012345

3 rows in set (0.00 sec)

Output 2: Flattened Dimensions and Specialization Catalog Output

product_id	product_name	category_name	seller_name	base_price	dimensions_lxwxh	specialized_properties
101	Noise Cancelling Headphones	Audio Devices	RetailNet India LLP	7999.00	18.00 x 15.00 x 7.50	Warranty: 24M, Power: 2.50W
102	Handspun Khadi Kurta	Ethnic Wear	Surat Fabric House	1499.00	75.00 x 48.00 x 2.00	Fabric: Khadi Cotton, Size: XL, Fit: Men
103	Filter Coffee Decoction 500ml	Grocery	South Agro Organics	350.00	10.00 x 8.00 x 22.00	Expiry: 2027-06-30, Veg: 1

3 rows in set (0.00 sec)

Output 3: Relational Join Output with Derived Calculations

order_id	customer_name	product_name	qty	unit_price	line_total	payment_mode	total_amount	courier_name	tracking_number	status
1001	Aarav Patel	Noise Cancelling Headphones	1	7999.00	7999.00	UPI	8699.00	BlueDart Express	BD-IN-9821334	DELIVERED
1001	Aarav Patel	Filter Coffee Decoction	2	350.00	700.00	UPI	8699.00	BlueDart Express	BD-IN-9821334	DELIVERED
1002	Diya Nair	Handspun Khadi Kurta	3	1499.00	4497.00	CREDIT_CARD	4497.00	Delhivery Logistics	DEL-IN-4455112	SHIPPED
1003	Vikram Iyer	Filter Coffee Decoction	4	350.00	1400.00	COD	1400.00	NULL	NULL	PLACED

4 rows in set (0.00 sec)

Output 4: Relational Table Inventory (SHOW TABLES;)

Tables_in_indian_ecommerce_db
addresses
categories
clothing_products
customer_phone_numbers
customers
deliveries
delivery_status_history
electronics_products
grocery_products
order_items
orders
payments
product_images
products
seller_contact_numbers
sellers

16 rows in set (0.01 sec)

Output 5: Referential Integrity Violation Demonstration Log

```
-- DEMONSTRATION (A): FOREIGN KEY VIOLATION (Customer 99 does not exist)
mysql> INSERT INTO orders (order_id, customer_id, status) VALUES (2001, 99, 'PLACED');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`indian_ecommerce_db`.`orders`, CONSTRAINT `fk_order_customer` FOREIGN KEY (`customer_id`)
REFERENCES `customers` (`customer_id`) ON DELETE RESTRICT ON UPDATE CASCADE)

-- DEMONSTRATION (B): UNIQUE CONSTRAINT VIOLATION (Duplicate email address)
mysql> INSERT INTO customers (first_name, last_name, email) VALUES ('Aarav', 'Sharma',
'aarav.patel@example.in');
ERROR 1062 (23000): Duplicate entry 'aarav.patel@example.in' for key 'customers.email'
```

```
-- DEMONSTRATION (C): COMPOSITE PRIMARY KEY VIOLATION ON WEAK/JUNCTION RELATION
mysql> INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (1001, 101, 2, 7999.00);
ERROR 1062 (23000): Duplicate entry '1001-101' for key 'order_items.uq_order_product'

-- DEMONSTRATION (D): STRICT 1:1 CARDINALITY VIOLATION (Duplicate foreign key on Order payment)
mysql> INSERT INTO payments (order_id, payment_mode, transaction_status, amount) VALUES (1001, 'NET_BANKING', 'SUCCESS', 8699.00);
ERROR 1062 (23000): Duplicate entry '1001' for key 'payments.order_id'

-- DEMONSTRATION (E): ON DELETE SET NULL VERIFICATION
mysql> INSERT INTO products (product_id, category_id, seller_id, product_name, length, breadth, height, base_price)
-> VALUES (104, 1, 1, 'Demo Category-Linked Gadget', 10.00, 10.00, 5.00, 999.00);
Query OK, 1 row affected (0.01 sec)

mysql> DELETE FROM categories WHERE category_id = 1;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT category_id, category_name, parent_category_id FROM categories WHERE category_id = 4;
+-----+-----+-----+
| category_id | category_name | parent_category_id |
+-----+-----+-----+
|          4 | Audio Devices |          NULL      |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT product_id, product_name, category_id FROM products WHERE product_id = 104;
+-----+-----+-----+
| product_id | product_name          | category_id |
+-----+-----+-----+
|          104 | Demo Category-Linked Gadget |          NULL |
+-----+-----+-----+
1 row in set (0.00 sec)
-- Observation: both categories.parent_category_id and products.category_id gracefully
-- shifted to NULL without throwing referential violation errors, confirming that
-- fk_cat_parent and fk_product_category are both correctly configured ON DELETE SET NULL.

-- DEMONSTRATION (F): ON DELETE CASCADE VERIFICATION
mysql> DELETE FROM customers WHERE customer_id = 9;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM customer_phone_numbers WHERE customer_id = 9;
Empty set (0.00 sec)

mysql> SELECT * FROM addresses WHERE customer_id = 9;
Empty set (0.00 sec)
-- Observation: Child records residing in customer_phone_numbers and addresses were automatically
purged.
```

Result and Discussion

1. Mapping Fidelity: The conceptual Entity-Relationship (ER) model representing the Indian E-Commerce platform was systematically translated into a fully normalized relational schema, adhering to all mathematical rules of relational synthesis.
2. First Normal Form (1NF) Resolution: The multi-valued attributes (phone_numbers on Customer, contact_numbers on Seller, images on Product, and status_history on Delivery) were successfully decomposed into separate relations with composite keys, removing repeating groups and multi-valued dependencies.

3. Composite Attribute Flattening: The composite attributes name (flattened into first_name, last_name) and dimensions (flattened into length, breadth, height) were mapped directly as scalar attributes in their parent relations.
1. Weak Entity Modeling: The weak entities Address and OrderItem were defined with composite unique/primary constraints incorporating their respective identifying parent keys (customer_id and order_id) alongside ON DELETE CASCADE actions.
2. Specialization Hierarchy: The is_a relationship between Product and its subclasses (Electronics, Clothing, Grocery) was implemented using the superclass-subclass table mapping strategy, ensuring clean domain modeling with minimal null storage.
3. Constraint Verification: Declarative integrity constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, ON DELETE CASCADE, and ON DELETE SET NULL) were verified using deliberate violation queries. Every constraint successfully protected the database against data corruption and orphaned records.